

Estructura de Datos - Evaluación Continua
Examen Teoría Primer Parcial - 25 de Marzo de 2022



POLITÉCNICA

Apellidos.....

Nombre..... DNI/Pasaporte:

ASIENTO.....



Ejercicio 1 (5 puntos).

Dado el TAD Pila de Números Enteros Positivos con las siguientes operaciones:

boolean vacia ();
void apilar (int x);
int desapilar ();

SE PIDE:

Codificar en Java el método **boolean comprobarOrdenacion** que, recibiendo al menos como argumento una *pila* perteneciente al TAD anterior, compruebe si sus elementos están ordenados de manera estrictamente ascendente, es decir, deberá devolver como resultado **true** si los elementos de la pila están ordenados ascendente (sin elementos repetidos), y **false** en cualquier otro caso.

EJEMPLOS:

- 1) Dada la *pila* mostrada en la figura1, el método deberá devolver como resultado *true*.
- 2) Dada la *pila* mostrada en la figura2, el método deberá devolver como resultado *false*.

1	1
3	2
5	8
6	7
7	9
9	
11	
23	
34	

Figura 1 Figura 2

OBSERVACIONES:

- En caso de que la *pila* se encuentre vacía el método devolverá *true*.
- Si la pila tiene elementos repetidos el método deberá devolver como resultado *false*.
- El método quedaría fuera de la clase Pila.

NORMAS DE OBLIGADO CUMPLIMIENTO (su no observación supondría la calificación de 0 puntos):

- **Queda expresamente prohibida la utilización de operaciones no definidas en el TAD.**
- **No se permite utilizar ninguna estructura de datos auxiliar.**
- **No se permite definir nuevos atributos en las clases.**
- Cada elemento de la *pila* solamente podrá ser, a lo sumo, desapilado y apilado una vez.
- Al finalizar la ejecución del método, la pila deberá quedar con el contenido y estructura iniciales.
- Responder sobre esta misma hoja. **No se permitirá la entrega de hojas adicionales.**

Solución:

```
private boolean comprobarOrdenacion(Pila pila, int anterior) {
    int elemento;
    boolean resultado = true;
    if (!pila.vacia ()) {
        elemento = pila.desapilar ();
        if (elemento > anterior)
            resultado = comprobarOrdenacion (pila, elemento);
        else resultado = false;
        pila.apilar (elemento);
    }
    return resultado;
}

public boolean comprobarOrdenacion(Pila pila) {
    return comprobarOrdenacion (pila, -1);
}
```

Estructura de Datos - Evaluación Continua
Examen Teoría Primer Parcial - 25 de Marzo de 2022



POLITÉCNICA

Apellidos.....

Nombre..... DNI/Pasaporte:

ASIENTO.....



Ejercicio 2 (5 puntos)

Sea un TAD lista ordinal de enteros implementado con una lista enlazada con las siguientes vistas públicas y privadas:

ListaOrdinal:

private Nodo inicio
private Nodo fin

Nodo:

public int getDato()
public Nodo getSiguiente()
public void setSiguiente(Nodo siguiente)

SE PIDE:

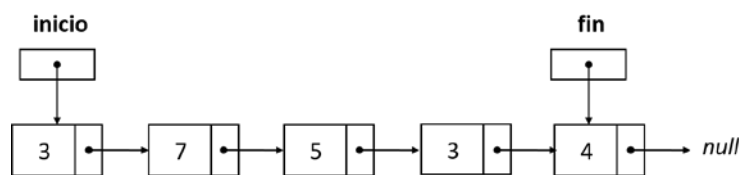
Codificar en Java el siguiente método de la clase **ListaOrdinal**:

public void moverAlFinal(int dato)

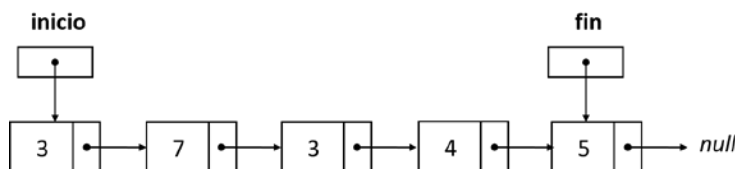
que mueva el dato que se pasa como parámetro al final de la lista sobre la que se ejecuta el método mediante un algoritmo iterativo.

EJEMPLO:

Sea la lista representada en la siguiente figura:



Al ejecutar el método con el dato 5 la lista quedará así:



OBSERVACIONES:

- Si la lista está vacía quedará vacía.
- Si el dato no se encuentra en la lista, esta quedará inalterada.
- Si el dato se encuentra varias veces en la lista, se moverá su primera aparición.
- No se pueden crear nuevos nodos.
- Se deberá realizar un único recorrido de la lista.

NORMAS DE OBLIGADO CUMPLIMIENTO (su no observación supondría la calificación de 0 puntos):

- **Queda expresamente prohibida la utilización de atributos u operaciones no definidas en ListaOrdinal o Nodo.**
- **No se permite utilizar ninguna estructura de datos auxiliar.**
- **No se permite definir nuevos atributos en las clases.**
- Responder sobre esta misma hoja. **No se permitirá la entrega de hojas adicionales.**

DURACIÓN TOTAL DEL EXAMEN: 1 hora 30 minutos
Fechas previstas: Publicación de resultados: 8-4-2022. Revisión de examen: 21-4-2022

Solución:

```
public void moverAlFinal(int dato) {
    Nodo actual = inicio;
    Nodo anterior = null;
    while (actual != null && actual.getDato() != dato) {
        anterior = actual;
        actual = actual.getSiguiente();
    }
    if (actual != null) {
        if (actual != fin) {
            if (actual == inicio) {
                inicio = actual.getSiguiente();
            } else {
                anterior.setSiguiente(actual.getSiguiente());
            }
            actual.setSiguiente(null);
            fin.setSiguiente(actual);
            fin = actual;
        }
    }
}
```